PROJET INFORMATIQUE — CONCEPTION & ARCHITECTURE

DOSSIER DE CONCEPTION MERISE & UML

Système de Réservation de Billets d'Avion (C++ / Qt & MySQL)

Document officiel de référence pour la modélisation sous Looping et Astah UML

> **Objectif du document :** Ce dossier synthétise l'intégralité de l'analyse et de la conception logicielle pour le développement du prototype de **Réservation de Billets d'Avion** en **C++ / Qt** avec base de données **MySQL**. Il fournit toutes les spécifications formelles nécessaires (Règles de gestion, MCD, MLD, MPD SQL, Use Cases UML, Diagramme de Classes, Séquence et État-Transition) ainsi que le guide pas à pas pour saisir manuellement les schémas dans les logiciels **Looping** et **Astah UML**.

1. Synthèse du Cahier des Charges & Règles de Gestion

L'application est un logiciel de bureau développé en C++ avec le framework Qt, destiné à gérer les réservations de vols en local sur une base MySQL. L'analyse des besoins permet d'identifier trois acteurs principaux et leurs prérogatives :

Acteur	Rôle & Responsabilités dans le Système	Portée des Droits
Visiteur	Consulter le catalogue des vols, rechercher un vol par ville départ/arrivée et date.	Lecture seule (Public)
Client	Créer un compte, réserver un billet pour un/plusieurs passagers, consulter/annuler ses réservations.	Espace Client sécurisé
Administrateur	Ajouter, modifier, supprimer des vols, gérer les places et suivre l'état de toutes les réservations.	Gestion Totale (Back-Office)

Règles de Gestion Métier (RG) :

• **RG01 (Unicité Vol) :** Chaque vol est identifié de façon unique par son numéro de vol (ex: AF-123).

• **RG02 (Coherence Trajet) :** La ville de départ doit être strictement différente de la ville d'arrivée.

• **RG03 (Contrainte Temporelle) :** La date et l'heure de départ d'un vol réservé doivent être dans le futur.

• **RG04 (Contrainte de Capacité) :** Une réservation est acceptée SSI *nombre_billets* <= *places_disponibles*.

• **RG05 (Mise à jour automatique Stock) :** Lors d'une réservation, *places_disponibles* = *places_disponibles* - *nombre_billets*. En cas d'annulation, les places sont réincrémentées.

• **RG06 (Référence Unique) :** Chaque réservation génère une référence unique alphanumérique (ex: REF-89X2Z).

• **RG07 (Calcul du Prix) :** Le montant total d'une réservation est calculé automatiquement : *prix_total = prix_unitaire_vol * nombre_billets*.

• **RG08 (Gestion Passagers) :** Une réservation est effectuée par un Utilisateur (Client) et enregistre la liste des passagers effectifs du voyage.

2. Modélisation MERISE de la Base de Données MySQL

Conformément à la méthode MERISE enseignée dans votre cours, la conception de la base de données passe par l'élaboration du **MCD** (Niveau Conceptuel), sa conversion en **MLD** (Niveau Logique Relationnel) et sa traduction en **MPD** (Script SQL MySQL).

2.1. Dictionnaire des Données

Code Attribut	Désignation / Signification	Type Conceptuel	Contrainte
id_utilisateur	Identifiant unique de l'utilisateur	Entier (AUTO_INC)	Clé Primaire
nom_util / prenom_util	Nom et prénom de l'utilisateur	Texte (VARCHAR 50)	Non Nul
email	Adresse email (identifiant connexion)	Texte (VARCHAR 100)	Unique, Non Nul
mot_de_passe	Mot de passe haché (SHA-256)	Texte (VARCHAR 255)	Non Nul
role	Rôle applicatif ('Client', 'Admin')	Texte (VARCHAR 20)	Default 'Client'
id_vol	Identifiant unique du vol	Entier (AUTO_INC)	Clé Primaire
numero_vol	Code du vol (ex: AF-456)	Texte (VARCHAR 20)	Unique, Non Nul
compagnie	Nom de la compagnie aérienne	Texte (VARCHAR 50)	Non Nul
ville_depart / ville_arrivee	Aéroports / Villes de trajet	Texte (VARCHAR 50)	Non Nul
date_depart / date_arrivee	Horaires prévus de vol	DateHeure (DATETIME)	Non Nul
prix	Prix unitaire d'un billet	Décimal (DECIMAL 10,2)	Positive (> 0)
places_disponibles	Nombre de sièges encore libres	Entier	Positif (>= 0)
id_reservation	Identifiant unique réservation	Entier (AUTO_INC)	Clé Primaire
reference	Code de confirmation unique	Texte (VARCHAR 20)	Unique, Non Nul
date_reservation	Horodatage de la réservation	DateHeure (DATETIME)	NOW()
nombre_billets	Nombre de places réservées	Entier	Strictement Positif (>0)
prix_total	Montant total de la commande	Décimal (DECIMAL 10,2)	Calculé
statut	État ('Confirmée', 'Annulée')	Texte (VARCHAR 20)	Default 'Confirmée'
id_passager	Identifiant unique du passager	Entier (AUTO_INC)	Clé Primaire
nom_pass / prenom_pass	Nom et prénom du passager	Texte (VARCHAR 50)	Non Nul
date_naissance	Date de naissance du passager	Date	Non Nul
numero_document	Passeport ou pièce d'identité	Texte (VARCHAR 50)	Optionnel

2.2. Modèle Conceptuel de Données (MCD)

Structure des Entités & Associations Conceptuelles :
• **ENTITÉ UTILISATEUR :** (id_utilisateur, nom, prenom, email, mot_de_passe, role)
• **ENTITÉ VOL :** (id_vol, numero_vol, compagnie, ville_depart, ville_arrivee, date_depart, date_arrivee, prix, places_disponibles)
• **ENTITÉ RESERVATION :** (id_reservation, reference, date_reservation, nombre_billets, prix_total, statut)
• **ENTITÉ PASSAGER :** (id_passager, nom, prenom, date_naissance, numero_document)

Associations et Cardinalités :
1. **Effectuer :** [UTILISATEUR] (0,N) <--- Effectuer ---> (1,1) [RESERVATION]
Règle : Un utilisateur peut effectuer 0 à N réservations. Une réservation est effectuée par 1 et 1 seul utilisateur.
2. **Concerner :** [VOL] (0,N) <--- Concerner ---> (1,1) [RESERVATION]
Règle : Un vol peut être concerné par 0 à N réservations. Une réservation concerne 1 et 1 seul vol.
3. **Inclure :** [RESERVATION] (1,N) <--- Inclure ---> (1,N) [PASSAGER]
Règle : Une réservation inclut 1 à N passagers. Un passager peut figurer dans 1 à N réservations.

2.3. Passage du MCD au MLD (Modèle Logique de Données)

Règles de Transformation Appliquées (selon votre cours) :
1. Chaque entité devient une **Table Relationnelle**.
2. L'association binaire `(1,1)-(0,N)` (*Effectuer* et *Concerner*) injecte la clé primaire de l'entité côté (0,N) comme **Clé Étrangère (#FK)** dans la table du côté (1,1) (Table `RESERVATION`).
3. L'association n-aire `(1,N)-(1,N)` (*Inclure*) se transforme en une **Table Intermédiaire Pivot** (`CONTIENT_PASSAGER`) dont la clé primaire est composée des deux clés étrangères.

Schéma Relationnel Textuel du MLD :
• **UTILISATEUR** (id_utilisateur, nom, prenom, email, mot_de_passe, role)
• **VOL** (id_vol, numero_vol, compagnie, ville_depart, ville_arrivee, date_depart, date_arrivee, prix, places_disponibles)
• **PASSAGER** (id_passager, nom, prenom, date_naissance, numero_document)
• **RESERVATION** (id_reservation, reference, date_reservation, nombre_billets, prix_total, statut, *#id_utilisateur*, *#id_vol*)
• **CONTIENT_PASSAGER** (*#id_reservation*, *#id_passager*)

2.4. MPD - Script SQL de Création MySQL

```
-- Base de données MySQL pour l'application C++ / Qt
CREATE DATABASE IF NOT EXISTS reservation_vol_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE reservation_vol_db;
-- 1. Table UTILISATEUR
CREATE TABLE UTILISATEUR (
    id_utilisateur INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'Client'
) ENGINE=InnoDB;
-- 2. Table VOL
CREATE TABLE VOL (
    id_vol INT AUTO_INCREMENT PRIMARY KEY,
    numero_vol VARCHAR(20) NOT NULL UNIQUE,
    compagnie VARCHAR(50) NOT NULL,
    ville_depart VARCHAR(50) NOT NULL,
    ville_arrivee VARCHAR(50) NOT NULL,
    date_depart DATETIME NOT NULL,
    date_arrivee DATETIME NOT NULL,
    prix DECIMAL(10,2) NOT NULL CHECK (prix > 0),
    places_disponibles INT NOT NULL CHECK (places_disponibles >= 0)
) ENGINE=InnoDB;
-- 3. Table PASSAGER
CREATE TABLE PASSAGER (
    id_passager INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    date_naissance DATE NOT NULL,
    numero_document VARCHAR(50)
) ENGINE=InnoDB;
-- 4. Table RESERVATION
CREATE TABLE RESERVATION (
    id_reservation INT AUTO_INCREMENT PRIMARY KEY,
    reference VARCHAR(20) NOT NULL UNIQUE,
    date_reservation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    nombre_billets INT NOT NULL CHECK (nombre_billets > 0),
    prix_total DECIMAL(10,2) NOT NULL,
    statut VARCHAR(20) NOT NULL DEFAULT 'Confirmée',
    id_utilisateur INT NOT NULL,
    id_vol INT NOT NULL,
    FOREIGN KEY (id_utilisateur) REFERENCES UTILISATEUR(id_utilisateur) ON DELETE CASCADE,
    FOREIGN KEY (id_vol) REFERENCES VOL(id_vol) ON DELETE CASCADE
) ENGINE=InnoDB;
-- 5. Table Pivot CONTIENT_PASSAGER
CREATE TABLE CONTIENT_PASSAGER (
    id_reservation INT NOT NULL,
    id_passager INT NOT NULL,
    PRIMARY KEY (id_reservation, id_passager),
    FOREIGN KEY (id_reservation) REFERENCES RESERVATION(id_reservation) ON DELETE CASCADE,
    FOREIGN KEY (id_passager) REFERENCES PASSAGER(id_passager) ON DELETE CASCADE
) ENGINE=InnoDB;
```

3. Modélisation Orientée Objet UML (Architecture C++ / Qt)

L'UML intervient pour structurer l'application C++ avec le framework Qt (IHM, contrôleurs, modèles métier et accès MySQL via QtSql).

3.1. Diagramme de Cas d'Utilisation (Use Cases UML)

Frontières du Système : Application de Réservation de Billets d'Avion (Qt Desktop)

• **Acteur Principal : Visiteur**
- *UC1 : Rechercher et consulter les vols* (par départ, destination, date).
- *UC2 : S'inscrire / Créer un compte*.

• **Acteur Principal : Client (hérite de Visiteur)**
- *UC3 : Se connecter / Se déconnecter*.
- *UC4 : Réserver un billet* <<include>> *Vérifier places disponibles*, <<include>> *Enregistrer Passagers*.
- *UC5 : Consulter ses réservations*.
- *UC6 : Annuler une réservation* <<extend>> *Restituer places libres*.

• **Acteur Principal : Administrateur**
- *UC7 : Gérer le catalogue des vols* (Ajouter, Modifier, Supprimer un vol).
- *UC8 : Consulter le tableau de bord* (Statistiques, liste globale des réservations).

3.2. Diagramme de Classes C++ / Qt (Architecture 3 Tiers)

L'architecture C++/Qt est découpée en 3 couches distinctes pour assurer une grande maintenabilité :

Couche Logicielle	Classes C++ / Qt	Rôle et Méthodes Clés
IHM / Présentation (Qt)	MainWindow DialogReservation AdminDashboard	Heritent de QMainWindow/QDialog. Gèrent l'affichage Qt (Qt Designer), les boutons, formulaires et signaux/slots (Q_OBJECT).
Logique Métier (C++ Pure)	Utilisateur Vol Passager Reservation	Classes C++ encapsulant les attributs privés, constructeurs, getters/setters et méthodes métier (ex: Vol::verifierDisponibilite(), Reservation::calculerPrixTotal()).
Accès Données (QtSql)	DatabaseManager	Singleton encapsulant `QSqlDatabase` et exécutant les requêtes préparées `QSqlQuery` (connexion MySQL, transactions SQL).

3.3. Diagramme de Séquence UML : Scénario 'Effectuer une Réservation'

Participants : Client (Acteur), :DialogReservation (IHM Qt), :ReservationController (Contrôleur), :DatabaseManager (DAO), :MySQL (SGBD)

1. **Client** -> :DialogReservation : Clic 'Confirmer la réservation' (saisie passagers)
2. :DialogReservation -> :ReservationController : creerReservation(idUtil, idVol, listePassagers)
3. :ReservationController -> :DatabaseManager : getPlacesDisponibles(idVol)
4. :DatabaseManager -> :MySQL : SELECT places_disponibles FROM VOL WHERE id_vol = ...
5. :MySQL --> :DatabaseManager : Retourne nbPlaces
6. **ALT (Si nbPlaces >= nbBillets) :**
a. :ReservationController -> :DatabaseManager : beginTransaction()
b. :DatabaseManager -> :MySQL : INSERT INTO RESERVATION ... & INSERT INTO PASSAGER ...
c. :DatabaseManager -> :MySQL : UPDATE VOL SET places_disponibles = places_disponibles - N
d. :DatabaseManager -> :MySQL : COMMIT
e. :ReservationController --> :DialogReservation : Succès (Affiche Référence)
7. **ELSE :**
a. :ReservationController --> :DialogReservation : Erreur ('Places insuffisantes')

3.4. Diagramme d'État-Transition : Cycle de Vie de l'Objet RESERVATION

• **État Initial :** [En Attente de Saisie]
• **Transition 1 :** [En Attente] -- *Evénement: Validation Formulaire [Garde: Places disponibles] / Action: Générer Référence & INSERT DB* --> **[Confirmée]**
• **Transition 2 :** [Confirmée] -- *Evénement: Demande Annulation Client / Action: UPDATE Statut='Annulée' & Réincrémenter Stock Vol* --> **[Annulée]**
• **Transition 3 :** [Confirmée] -- *Evénement: Vol Effectué / Action: Clôturer dossier* --> **[Terminée]** (État Final)

4. Guide de Reproduction sous Looping & Astah UML

Pour obtenir la note maximale lors de l'évaluation, vous devez saisir manuellement ces modèles dans vos logiciels de modélisation. Voici les instructions pas à pas :

4.1. Saisie du MCD dans le logiciel LOOPING

• 1. Ouvrez **Looping** et créez 4 Entités : UTILISATEUR, VOL, RESERVATION, PASSAGER.

• 2. Pour chaque entité, ajoutez les attributs (Cochez *Identifiant* pour `id_utilisateur`, `id_vol`, `id_reservation`, `id_passager`).

• 3. Tracez l'association Effectuer entre UTILISATEUR (cardinalité 0,N) et RESERVATION (cardinalité 1,1).

• 4. Tracez l'association Concerner entre VOL (cardinalité 0,N) et RESERVATION (cardinalité 1,1).

• 5. Tracez l'association Inclure entre RESERVATION (cardinalité 1,N) et PASSAGER (cardinalité 1,N).

• 6. Cliquez sur l'onglet **MLD** : Looping génère automatiquement le schéma relationnel et la clé étrangère `#id_utilisateur` et `#id_vol` dans RESERVATION !

• 7. Cliquez sur l'icône **SQL** pour exporter le script `CREATE TABLE` MySQL.

4.2. Saisie des Diagrammes dans ASTAH UML

• 1. **UseCase Diagram :** Créez le cadre système *'Système Réservation'*. Placez les acteurs Visiteur, Client (flèche d'héritage vers Visiteur) et Administrateur. Reliez les bulles de Use Cases avec les dépendances <<include>> et <<extend>>.

• 2. **Class Diagram :** Créez les paquets IHM, Metier, DAO. Dans Metier, ajoutez les classes C++ avec leurs attributs privés (-) et leurs méthodes publiques (+). Tracez les associations de composition (losange plein).

• 3. **Sequence Diagram :** Créez les lignes de vie (Client, DialogReservation, ReservationController, DatabaseManager, MySQL). Tracez les messages synchrones et utilisez le bloc alt/else pour la vérification de disponibilité.

• 4. **Statechart Diagram :** Créez les états En Attente, Confirmée, Annulée, Terminée et reliez-les par les évènements de transition.

> **Prochaine Étape Recommandée : Vous avez désormais l'ensemble de la conception théorique et technique. Vous pouvez commencer directement la saisie dans Looping et Astah, exécuter le script SQL dans votre SGBD MySQL (via phpMyAdmin ou Workbench) et attaquer le développement de l'interface graphique sous Qt Creator (C++) !**